

# nanogpt\_speedrun\_lessons\_1

What two autonomous speedrun campaigns produced, what they wasted, and the operating procedure a third should start from

Claude Opus 5 (OPHIS) · for baiyuzhu@mit.edu

2026-08-01

## Contents

|                                                                                      |           |
|--------------------------------------------------------------------------------------|-----------|
| <b>0. What this document is</b>                                                      | <b>2</b>  |
| <b>1. The two frames, and why the difference dominates everything</b>                | <b>2</b>  |
| 1.1 v3 — step-matched . . . . .                                                      | 2         |
| 1.2 reimplementatation — time-matched . . . . .                                      | 3         |
| 1.3 The lesson . . . . .                                                             | 3         |
| <b>2. The scoreboard</b>                                                             | <b>3</b>  |
| 2.1 v3 — what won (step-matched) . . . . .                                           | 3         |
| 2.2 reimplementatation — what happened (time-matched) . . . . .                      | 4         |
| 2.3 The uncomfortable comparison . . . . .                                           | 4         |
| <b>3. Mechanisms that survived — the portable physics</b>                            | <b>5</b>  |
| 3.1 Fill-flood (throughput) . . . . .                                                | 5         |
| 3.2 The memoriser is the model . . . . .                                             | 5         |
| 3.3 The data-repetition cliff . . . . .                                              | 5         |
| 3.4 The capacity lever changes sign . . . . .                                        | 6         |
| <b>4. What went RIGHT — keep all of this</b>                                         | <b>6</b>  |
| <b>5. What went WRONG — the failure taxonomy</b>                                     | <b>7</b>  |
| 5.1 Drilling a saturated frame (largest cost by far) . . . . .                       | 7         |
| 5.2 Running a filter instead of a research programme . . . . .                       | 8         |
| 5.3 Under-powered decisions at a threshold . . . . .                                 | 8         |
| 5.4 Selection and confirmation sharing draws . . . . .                               | 8         |
| 5.5 Contention on a shared box . . . . .                                             | 8         |
| 5.6 The screen was wrong three times . . . . .                                       | 9         |
| 5.7 Specification errors — the knob did not reach the parameters . . . . .           | 9         |
| 5.8 Process errors that cost real time . . . . .                                     | 10        |
| 5.9 The meta-failure . . . . .                                                       | 10        |
| <b>6. The research loop (the procedure that works)</b>                               | <b>10</b> |
| <b>7. The paper requirement</b>                                                      | <b>11</b> |
| 7.1 Next Experiments is the paper’s primary output . . . . .                         | 11        |
| 7.2 Every proposed experiment uses exactly these four headings, as bullets . . . . . | 11        |
| 7.3 A paper is not written until the PDF builds . . . . .                            | 11        |

|                                                 |    |
|-------------------------------------------------|----|
| 8. Statistics and reporting conventions         | 11 |
| 9. Hard rules — the checklist                   | 12 |
| 10. Seeding the new project                     | 12 |
| 10.1 What to carry over . . . . .               | 12 |
| 10.2 Day-one checklist . . . . .                | 13 |
| 10.3 The one strategic recommendation . . . . . | 13 |
| 11. Open questions worth inheriting             | 13 |
| 12. Closing                                     | 14 |

## 0. What this document is

Two autonomous research campaigns have now run against the nanoGPT-speedrun problem in /Users/baiyu/Desktop/OPHIS:

| project                           | frame                                       | outcome                                                                 |
|-----------------------------------|---------------------------------------------|-------------------------------------------------------------------------|
| vibeautoresearch-baiyu-v3         | <b>step-matched:</b> 2000 steps, SDPA       | baseline 0.930647 → <b>0.922710</b> , four additive wins                |
| vibeautoresearch_reimplementation | <b>time-matched:</b> 300 s on one H200, FA3 | baseline 0.931857 → <b>no adopted result</b> ; one real sub-gate effect |

Together they logged well over a thousand experiments, 33 papers, ~2600 lines of hourly self-assessment, and seven integrity findings that each retracted or reversed a conclusion. This document is the distillation: **what to keep, what to never repeat, and the exact procedure a third campaign should start with on day one.**

It is written for an agent, not for a reader who already knows the history. Every rule below is traceable to a specific failure. None of them are aspirational.

**How to use it.** Sections 1–3 are the factual record (read once). Sections 4–5 are the lessons (read once, carefully). **Sections 6–9 are the operating procedure — those are the ones to keep open while working.** Section 10 is the day-one checklist for the new repo.

## 1. The two frames, and why the difference dominates everything

The single most consequential fact across both campaigns is that **the frame determines which mechanisms can possibly win**, and the two projects had different frames.

### 1.1 v3 — step-matched

```
STOP_MODE=steps MAX_STEPS=2000 ATTN_BACKEND=sdpa
shards 1-10 train / 6542 val
_seed 0.000847 → 2 adoption gate 0.001694
```

Every arm gets the **same optimizer steps and the same training tokens**. An intervention that costs 20% more time per step is not penalised at all in the metric. Quality and cost are separate axes, reported separately.

**Consequence: architecture wins are visible.** Adding capacity, adding an attention mask, adding an n-gram order — all of these cost time and none of that cost registers. v3 found four wins totalling  $-0.0079$  and they were all of this kind.

## 1.2 reimplementation — time-matched

```
STOP_MODE=time TIME_BUDGET=300 ATTN_BACKEND=fa3
COMPILE_MODE=max-autotune-no-cudagraphs WINDOW_PATTERN=TTL
DEVICE_BATCH_SIZE=72 TOTAL_BATCH_SIZE=147456
ten frozen shards (~250M unique tokens)
0.001307 → 2 adoption gate 0.002614, min_seeds = 10
baseline 0.931857
```

Every arm gets the **same wall clock**. Now  $\text{tokens} = \text{throughput} \times 300 \text{ s}$ , so anything that costs time costs tokens, and tokens are the dominant term:

$$\text{val\_bpb} \approx a - b \ln(\text{tokens}), \quad b \approx 0.06$$

**Consequence: the same architecture wins become losses.** A mask that costs 20% step time must buy back  $0.06 \times \ln(1.2) = 0.011$  bpb of intrinsic quality just to break even — roughly **4× the entire adoption gate**. Almost nothing does that.

## 1.3 The lesson

**Choosing the frame is the highest-leverage decision in the whole campaign, and it is made before any experiment runs.** A step-matched frame rewards adding structure. A time-matched frame rewards removing it. The same idea has opposite sign in the two frames, and neither answer is wrong — they answer different questions.

If the new project’s real objective is *wall-clock speedrun*, use the time-matched frame and accept that the architecture space is mostly closed. If the objective is *quality at fixed compute-steps*, use step-matched and accept that the results do not transfer to a wall-clock leaderboard.

**Write the frame’s arithmetic table into the repo on day one** and re-read it before proposing anything:

| axis                                           | value                                |
|------------------------------------------------|--------------------------------------|
| dense params                                   | 94M                                  |
| n-gram table params                            | 2.82e9 ( <b>96.8% of the model</b> ) |
| unique tokens                                  | ~250M                                |
| tokens consumed                                | 300–500M (~2 passes)                 |
| budget                                         | 300 s, one H200                      |
| adoption gate                                  | 0.002614                             |
| <b>intrinsic (non-throughput) effect scale</b> | <b>~1e-3</b>                         |
| <b>throughput effect scale</b>                 | <b>~1e-2</b>                         |

The last two rows are the ones that kill proposals. An intrinsic knob has ~0 expected effect *and* ~0 resolvable information against a 2.6e-3 gate. That arithmetic was available from early on and was ignored for weeks.

## 2. The scoreboard

### 2.1 v3 — what won (step-matched)

| # | lever                   | mechanism                                                | val_bpb         | $\Delta$ |
|---|-------------------------|----------------------------------------------------------|-----------------|----------|
| 1 | NGRAM_TABLE_MULT=256    | hash-bucket capacity for rare bigram/trigram VEs         | 0.925318        | -0.00405 |
| 2 | DOC_MASK_MODE=seg       | block cross-document attention in packed rows            | 0.923930        | -0.00139 |
| 3 | WARMDOWN_RATIO=0.75     | earlier/longer LR warmdown                               | 0.923684        | -0.00040 |
| 4 | NGRAM_FOURGRAM_MULT=256 | 4th order context adds information beyond bigram+trigram | <b>0.922710</b> | -0.00131 |

Verified on 6 hold-out seeds. Additivity was **measured, not assumed** (the seg-mask marginal survives on top of capacity,  $t = -5.38$ ).

**The organising insight, which held across ~30 experiments on 8 axes:**

**Wins come from ADDING capacity, structure, or schedule that does not disturb the tuned optimizer. Every model/optimizer PERTURBATION regressed.**

SwiGLU +0.0041 · residual scaling +0.0142 · cautious Adam +0.0111 · shared-trigram VE +0.0016 · window+seg +0.0010 · tied embeddings **+1.78 (catastrophic)** · RoPE reset null · VE-LR  $\times 0.5$  null · Demon 1 null · WD 0.15 null · zero-init lm\_head null.

The frozen config (Muon/NorMuon + QK-norm + squared-ReLU + n-gram VE) is a tight co-tuned local optimum. Poking it loses. Bolting things onto it can win.

## 2.2 reimplementing — what happened (time-matched)

- **1109 logged experiments**, 46 blocks, 33 papers, 31 directions closed.
- Best measured val\_bpb **0.927183**; baseline 0.931857.
- **No adopted result.**
- One real effect: **depth 6 (dim 640) + MATRIX\_LR 0.03**
  - $n = 97$  paired seeds, **91/97 better**, mean **-0.002431**, sd 0.001532, sem 0.000156,  $t = -15.62$ , 95% CI [-0.002736, -0.002126]
  - **0.93 gates** against a 0.002614 floor  $\rightarrow$  real, precisely determined, **sub-gate**, threshold test inconclusive.
- Three mechanisms confirmed, two of the agent’s own explanations refuted, one adoption claimed and **retracted**, measurement apparatus rebuilt four times.

## 2.3 The uncomfortable comparison

v3 found -0.0079 in ~30 experiments. The reimplementing found nothing adoptable in ~1100. **That is not a competence gap, it is the frame (§1.3)** — but it is also partly a discipline gap, and §5 is about which part.

### 3. Mechanisms that survived — the portable physics

These four are the transferable scientific content. Each made a prediction *before* the data existed and the prediction held.

#### 3.1 Fill-flood (throughput)

**Phenomenon.** MFU sits at ~19% while individual block GEMMs run at ~90%.

**Mechanism.** The hashed n-gram value embeddings hold ~1.61e9–2.82e9 parameters. A forward pass gathers at most  $B \times T \approx 1.5e5$  rows per site — a ~1e-4 fraction. Autograd’s **dense** embedding backward allocates a gradient the size of the *whole table* and must zero it, emitted as 194k tiny fill kernels. **Fill launches scale with TABLE size, not with TOUCHED-ROW count.**

**The discriminating prediction.** A sparse-gradient replacement should get *better* as tables grow — which a bandwidth-bound account **forbids**. Registered before the data. Penalty fell from +12.95% to +3.22% as tables grew 4×, and an out-of-sample third point at  $m=128$  landed at +15.8 ms against +14.9 predicted,  $R^2 = 0.995$ .

**Status: confirmed, direction closed on economics.** Break-even is near  $m \approx 343$ , and  $m=256$  already costs 4.2 gates of quality. A real mechanism with **no reachable operating point**.

#### 3.2 The memoriser is the model

**Phenomenon.** Depth 5 reaches the **lowest** train loss of any depth (2.4927) and the **worst** val loss (2.7722).

**Mechanism.** The n-gram table is 96.8% of the model. In a hashed table, a rare-key memorisation and a frequent-key statistic occupy rows drawn from **one hash space**, and an  $L_2$  penalty is indifferent between them.

**Evidence.** Table decay closes 93% of the train/val gap ( $+0.2795 \rightarrow +0.0201$ ) and the **endpoint still degrades**, at both depths.

**Status: confirmed. This closes the entire weight-space-regularisation family**, not one knob — which makes it worth more than the negative result that produced it. One member survives untested: a penalty that separates the populations by **visit count** rather than by norm.

#### 3.3 The data-repetition cliff

**Phenomenon.** val\_bpb collapses 16–18 gates past ~2.3 passes over the frozen corpus. Shape-independent across 39.8M–94.4M dense params:

| configuration      | tokens | passes | val_bpb       |
|--------------------|--------|--------|---------------|
| depth 6, dim 640   | 483M   | 1.9    | <b>0.9295</b> |
| 2× budget, depth 8 | 578M   | 2.3    | 0.9870        |
| depth 6, dim 512   | 623M   | 2.5    | 0.9729        |
| depth 5, dim 512   | 668M   | 2.7    | 0.9784        |

**The confound that had to be broken.** Shard order was fixed, so “the third pass” and “the tail of the corpus for the third time” were perfectly confounded in *every prior run*. Shuffling row-group order separates them: it moved the endpoint 0.0022 — 4.7% of the 0.0464 cliff — against preregistered thresholds of <0.003 (repetition) and >0.01 (corpus region). **The cause is repetition.**

**v3 measured the same wall from the other side.** Exact n-gram repeat statistics over the 294.9M-token stream:

| context | distinct | singleton rate | median occurrences |
|---------|----------|----------------|--------------------|
| bigram  | 8.49M    | 1.2%           | 1468               |
| trigram | 70.4M    | 15.9%          | 26                 |
| 4-gram  | 164.8M   | 45.8%          | 2                  |
| 5-gram  | 229.0M   | <b>69.9%</b>   | <b>1</b>           |

Going 4→5-gram, the fraction of positions whose context **never recurs** jumps to 69.9% and the median context is seen **exactly once**. A hashed row for a singleton context receives one gradient in the entire run and cannot generalise. **The 5-gram rung is starved by construction at this token budget** — a data-budget artifact, not an architectural limit.

**Both campaigns independently hit the same wall: the corpus is the binding constraint.** v3 hit it as an order ceiling; the reimplementer hit it as a repetition cliff. Neither could test the fix, because the corpus is frozen in both scope keys.

### 3.4 The capacity lever changes sign

Cutting hash rows costs **+0.0079** below the cliff and gains **−0.0163** (6.2 gates) above it. Crossing depth showed the flip tracks the **train/val gap**, not the pass count: capacity reduction helps exactly where the model overfits and nowhere else — i.e. only in configurations that are already 16–18 gates uncompetitive. **The second mechanism with no reachable operating point.**

## 4. What went RIGHT — keep all of this

**1. Preregistration actually worked.** Every confirmed mechanism in §3 made a numeric prediction before the data existed, and the two that failed were killed by their own preregistered falsifiers. Fill-flood’s  $R^2=0.995$  out-of-sample point is the strongest single artifact either campaign produced. **Keep: no run launches without a written prediction and a numeric kill criterion.**

**2. Retraction was executed, not just promised.** An adoption was claimed at 1.04 gates and **retracted two blocks later** with a heading of equal prominence in `SOTA_LOG.md`, because the rule “selection and confirmation never share draws” was fixed *before* the data. That rule caught the error it was written for.

**3. The agent refuted its own explanations, twice.** The “duration” and “dilution” accounts of the null refinements were both proposed by the agent and both killed by the agent’s own experiments. The null was then recorded as **unexplained** rather than assigned to the last surviving guess. That is the single healthiest habit in the record.

**4. Mechanical controls beat written rules.** Every recurring error was fixed *twice* — once with a rule that failed, once with a mechanism that worked:

| error                                         | failed fix          | working fix                                                                                                                                                                                                                                                                               |
|-----------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| invented enum values (×4)                     | “check the enums”   | <code>tools/registry_authoring.py</code><br>— <code>seal()</code> refuses to fingerprint an invalid record; <code>append()</code> serialises <b>all</b> records before writing <b>any</b> write text to a file, hand the file to the tool ( <code>git commit -F, scp</code> the launcher) |
| shell quoting broke<br>commits/launchers (×5) | “escape carefully”  | <code>probe_gate.sh</code> — a 40 s <b>real training job</b> per candidate GPU                                                                                                                                                                                                            |
| GPUs picked from <code>nvidia-smi</code>      | “check utilization” |                                                                                                                                                                                                                                                                                           |

| error       | failed fix    | working fix                                                  |
|-------------|---------------|--------------------------------------------------------------|
| SSH outages | retry by hand | <code>tools/ssh_retry.sh</code> — 8 attempts, linear backoff |

**This is the most portable lesson in the document.** A rule an agent must remember is a rule that will be broken under time pressure. A tool that refuses the bad action cannot be.

**5. The integrity gate caught what the record-level validator could not.** While writing this document, the agent backfilled mechanism records whose `observable_ids` were plausible-looking prose labels. `seal()` accepted them — each record was internally valid. The repo-level gate rejected them: **references missing observable** `obs_four_row_collapse`. **The schema forces every prediction to bind to an instrument that actually exists.** You cannot predict what you cannot measure, and the gate enforces it. Keep this two-layer design: record-level validation *and* cross-registry reference checking.

**6. Smoke tests saved four tranches.** A `NameError` from a generator defined inside an untaken branch; a `DEPTH` variable used before definition; an OOM from a 10.5 GiB gradient; an `nn.ModuleDict` reserved-key crash. Each caught before GPU time. **Always smoke-test the exact launcher on 30 steps before the tranche.**

**7. Executing beats inspecting.** v3’s integrity findings 1, 5 and 6 share one pattern: *verification that inspects rather than executes*. An uninitialized gate that looked initialized (**fourgram\_gate, 10 runs retracted, a false negative that later became SOTA #4**); four SOTA snapshots that all passed hash verification and **none of which could run** (each omitted `observable.py`); a validator call reading `o.get('errors')` when the validator emits **error**, so several “validate clean” confirmations — **including one in a commit message** — were vacuous. **Run the thing. Hashes validate the files that are present and say nothing about a file that was never included.**

## 5. What went WRONG — the failure taxonomy

Ordered by cost. Each has a rule attached; §9 collects the rules.

### 5.1 Drilling a saturated frame (largest cost by far)

Eleven levers in one session, **zero adoptions**: `DEVICE_BATCH_SIZE`, `NGRAM_TABLE_MULT`, `NGRAM_STATE_ROWWISE`, `VAL_LOSS_EVERY`, `COMPILE_MODE`, `MUON_NS_STEPS`, `NGRAM_VE_LR_SCALE`, `MUON_MOMENTUM_CONTINUOUS`, `WARMDOWN_RATIO`. Every one a **knob**. The session’s own measurements showed intrinsic effects live at  $\sim 1e-3$  against a  $4.07e-3$  gate.

**The arithmetic said no knob could clear the gate, and the knobs kept being launched anyway** — because knobs are cheap, always produce a number, and each null *feels* like ruling something out. It is not: at  $\pm 2$  single-seed band  $\pm 0.00122$ , a small real knob gain is **invisible even if real**. A knob has 0 expected effect **and** 0 resolvable information.

**No knob has ever cleared the gate in either campaign.** Every large win came from a structural change: FA3 ( $-0.0119$ ), warm `torch.compile` ( $-0.0439$ ), varlen doc-masking, block-sparse segmentation.

**RULE.** Classify every proposal as **KNOB** (existing flag, no new code) or **MECHANISM** (new code that changes what the model or kernel does) *and write the classification in the block’s paper*. After **three consecutive blocks with zero adoptions**, the next block is **MECHANISM-only**. Hard gate.

## 5.2 Running a filter instead of a research programme

Of 123 external claims ingested, **7 ever did predictive work**. Fifteen hours of experiments ran before the most scope-relevant paper was opened. Reading Engram (arXiv 2601.07372) — flagged as a *priority mechanism to implement* — revealed that `train.py` already implements all five of its architectural features. The 1.61e9 n-gram parameters *are* Engram. An hour of reading would have removed that entry from the priority list.

**RULE.** After **every** experiment block, before proposing the next, read at least one **new primary source** (WebSearch/WebFetch, 2025–2026 preferred) and write the judgement down **in scope arithmetic**, yes or no. A paper whose smallest model is 18× ours and whose shortest budget is 100× ours is not evidence about this frame, however good the method. **A recorded “no, and here is the arithmetic” is as valuable as a yes** — it is what stops the corpus filling with inert claims. Also: **check whether we already have it**.

## 5.3 Under-powered decisions at a threshold

`min_seeds=10` was adequate for rejecting large effects and **inadequate for resolving marginal ones**. The same configuration read 1.04, 0.60, 1.07, 0.83, 0.95, 1.09 gates across six disjoint 10–30 pair blocks. At `n=10` this frame cannot distinguish 0.60 gates from 1.07.

Worse — and this is the subtlest error in the record — **adding data made the threshold test harder**. At `n=78` the mean was  $-0.002311$  and the calculation said 97 pairs would exclude the floor. The better-determined mean at `n=97` is  $-0.002431$  — *closer* to the floor — so the gap the interval must clear shrank from 0.000303 to 0.000183 and **the requirement rose from 97 to 269 pairs**.

The power calculation held the mean fixed. **A power calculation that ignores uncertainty in the effect size systematically understates what is needed near a boundary**.

**RULE.** Before spending seeds to settle a threshold question, compute the requirement **with the effect size treated as uncertain**. If the effect sits within ~1 sem of the floor, the honest answer is “this frame cannot settle it” — say that instead of buying more seeds.

## 5.4 Selection and confirmation sharing draws

E28 cleared at 1.04 gates on seeds 42–51 — the same seeds used to *select* depth 6 and `MATRIX_LR` 0.03. Verification on disjoint seeds read 0.60. Adoption retracted.

The arithmetic:  $E[\max \text{ of } k \text{ standard normals}] = 1.029$  at  $k=4$ , so a 4-arm sweep at `n=4` inflates its winner by **0.26 gates**, additive across stacked selections.

**But the follow-up over-explained it.** A reflection attributed E29’s low reading to selection bias — and then E31, equally disjoint, read **1.07**. One disjoint block low, one high; between-block sd 0.000690 against 0.000459 expected. **The spread was noise, and the agent had attributed it to a mechanism because it had just computed a number that matched.** A correction notice was appended.

**RULE 1.** Selection and confirmation never share draws. **RULE 2.** Discount a swept winner by ~0.26 gates per selection at  $k=4$ , `n=4` before comparing it to the gate. **RULE 3.** Do not explain a single block’s deviation. One `n=10` block is too noisy to attribute to anything.

## 5.5 Contention on a shared box

The box is shared. **With all five candidate GPUs reporting 0% utilization, a 40 s training probe on each returned 233–237 ms against a 149.9 ms reference — a uniform 1.58× box-wide slowdown, equal across every device.** A per-GPU utilization counter cannot see that, because the bottleneck is shared.

A uniform slowdown is **not noise to be screened out afterwards — it is a different experiment**. At 237 ms a depth-8 arm completes ~1265 steps instead of ~2016, so the pair answers “these two models at half

the token budget”, where depth 6 is no longer near the data wall. **No post-hoc correction recovers the intended comparison.**

Two settling tranches were destroyed by co-tenant bursts (12 of 30 pairs lost, 19 of 20 lost). One earlier fp8 “win” turned out to be a **cross-GPU contention artifact**.

**RULES.** Probe-gate every launch with a real training job. Interleave treatment and control across GPUs *within* each batch, or run same-GPU sequential. Leave 1–2 GPUs free. Never poll the remote with one-off SSH.

## 5.6 The screen was wrong three times

Each version was replaced only after it let a bad arm through, and each time the arm changed a verdict.

| version                          | blind spot                                                                                                      |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------|
| absolute step floor (steps 1950) | assumes one operating point — would have quarantined a healthy <code>mult=256</code> cell finishing ~1620 steps |
| median step time alone           | sees steady state only — missed an arm losing ~13% of wall clock <i>outside</i> the timed steps                 |
| duty cycle alone                 | <b>scale-invariant</b> — a uniform slowdown leaves duty 1.0 while the arm sees half the tokens                  |

The duty-cycle failure was expensive: E11 seed 51 had duty 1.025, median step 192.4 ms against a cell median of 91.8 (2.11 $\times$ ), completed 1598 steps against ~3270, **passed the screen**, and at n=7 moved the paired mean from  $-0.0027$  to  $+0.0005$  *by itself*.

**Current screen (two-sided, both conditions required):**

```
duty = steps / (TIME_BUDGET_ms / median_step_ms)
clean iff duty >= 0.95 AND median_step_ms <= 1.15 * CLEAN_REFERENCE_median
```

And a fourth failure found later: **when contamination is the majority, the cell median is corrupted too**. A burst contaminated 19 of 20 pairs, so the in-tranche median was itself a burst value (~207 ms for a cell whose true median is 91.8) and **every contaminated arm would have passed**. Compare against a **stored clean reference per configuration**, not the tranche’s own median.

| configuration              | clean reference median |
|----------------------------|------------------------|
| depth 8, dim 768 (control) | 149.9 ms               |
| depth 6, dim 640           | 91.8 ms                |
| depth 5, dim 512           | 64.2 ms                |

**RULE.** Every screen is a proxy that silently assumes something is held constant. **Before adopting the next one, ask what it assumes and construct the arm that violates it.** Report both numbers for every arm **at collection time, before looking at val\_bpb**.

## 5.7 Specification errors — the knob did not reach the parameters

**E14** swept `WEIGHT_DECAY`, which reaches 3.2% of the model, while attributing the result to the n-gram tables, which are 96.8% and are governed by `NGRAM_WD_LAMBDA`. The experiment could not have answered its own question.

**The offset-augmentation null** is the same shape in reverse: the source omits whether the label offset is drawn per token or per sequence; the agent chose per token, which makes the target a mixture over offsets and the Bayes-optimal output a blur **by construction**. The method was reported as failed. It is **untested, not refuted**.

**RULE.** Before launch, **trace the knob to the parameters it modifies** and state the fraction of the model it touches. If an external method has an unspecified detail, name the choice you made and why — a guess that makes the target unidentifiable is a specification error on your side, not a refutation.

## 5.8 Process errors that cost real time

- **Uploaded `train.py` mid-tranche.** The worst available error. Reverted within one action and all 16 arms proved clean by `grep -c TNGRAM_SHARED` — a direct observable, not a timestamp argument.
- **Reported “confirmed at `n=4`”** when only 1 seed was clean in all three arms. → *State  $n$  **after** the screen, never from the raw arm list.*
- **Power analysis tested the CI upper bound instead of the lower**, so every row printed “EXCLUDES” while contradicting the straddle stated above it.
- **`$(d/512)`** sent two width smokes to the same GPU; timings discarded.
- **`research/ideas/` went 30 hours stale** at the end of the campaign while the log, state file and chart stayed current. Both dropped artifacts were the ones **no automated consumer reads**. → *An artifact with no automated consumer needs an explicit checklist item, or it silently stops being written.*

## 5.9 The meta-failure

The campaign ran a filter, not a research programme.

It was very good at *rejecting* things (31 directions closed, correctly) and poor at *generating* things worth testing. The structural explanation — the frame is a boundary optimum — is well-evidenced and also **exactly what an agent would say if it had simply searched a saturated frame for too long**. Both are true. The honest version: **saturation should have been recognised around direction 12, not direction 31, and the response should have been a new frame rather than a 20th refinement**.

---

## 6. The research loop (the procedure that works)

Per block of at most 5 experiment rounds:

1. **Write the block’s paper first.** It is a **precondition** — no experiment in the block runs until the paper exists. See §7.
2. **Generate ideas from several independent mechanistic lenses** — throughput / kernels, timed-region & data pipeline, quality-per-token, literature→concrete. Ground every idea in the actual code and screen against the already-tried set.
3. **Verify each idea against the code yourself.** Do not trust a subagent’s word. Dedupe; rank by expected-effect  $\times$  tractability  $\times$  risk.
4. **Adversarial critic before any GPU time.** Run a critic over the pooled proposals; it sets the execution order and the per-lever gate. **Record what the critic killed and why — a proposal set with no casualties means the critic was weak.** It has caught a  $4\times$  arithmetic error from a stale `vocab_size` default and a silently reverted n-gram capacity.
5. **Author the chain:** idea → mechanism → hypothesis → intervention → gated experiment, each with a prediction and a preregistered gate. `validate` and `audit` must be green **before** launching.
6. **Smoke-test the exact launcher** at  $\sim 30$  steps.
7. **Probe-gate the GPUs**, then launch **one tranche at a time**.
8. **Screen every arm two-sided against the clean reference, before looking at `val_bpb`.** Quarantine with the reason recorded.
9. **Record honestly** — negatives, segfaults and drift are recorded as failures, not hidden. Update the chart and the hourly log after **every** experiment. Commit and push.
10. **On a real SOTA** (clears the gate through the full funnel on disjoint seeds): snapshot code + `repro.json`, commit, push, republish chart, **and then run the snapshot standalone to prove it**

executes.

---

## 7. The paper requirement

Every block gets a paper in AI\_papers/ before it runs. Naming: paper\_<NNN>\_<slug>\_<date>.

### 7.1 Next Experiments is the paper’s primary output

A paper that only reports conclusions is **incomplete and must not be committed**. Retrospective analysis exists to justify the proposals. A “future work” paragraph that is vague has failed the paper’s purpose.

### 7.2 Every proposed experiment uses exactly these four headings, as bullets

- **Claim** — the specific external or internal claim being drawn on, cited by registry ID, with its recorded scope and evidence class. If no claim backs it, **say so explicitly**; a self-proposed idea is allowed but must be *labelled* rather than dressed up as literature.
- **Mechanism** — the physics-style causal account: name the **observed phenomenon** first, then the proposed cause, then the chain from cause to the measured endpoint. *A mechanism that only restates the intervention is not a mechanism. State what a rival account would predict differently, and which observable separates them. If nothing separates them, it is not yet testable.*
- **Hypothesis** — the falsifiable prediction **with numbers**: estimand, direction, magnitude, seed count, decision rule, and the **trap-check that kills the round early**. Fix the analysis rule here, before launch — including whether the token-law control variate applies (**it does NOT when the treatment’s own mechanism is throughput**).
- **Reasoning** — why this should work *here*, in this frame, given what has already been measured. **Name the prior evidence for AND against**. If a previous result argues against the proposal, that argument goes in this section, not omitted.

Then rate 1–5 on **novelty, provenance, validity, impact, reliability, feasibility, falsifiability**, each with a one-line evidence-based rationale. Ratings come **after** the four headings, not instead of them.

### 7.3 A paper is not written until the PDF builds

```
cd AI_papers && tectonic -X compile paper_0NN_*.tex      # or: pandoc x.md -o x.pdf
```

A .tex that has never compiled has never been proofread. Commit source and PDF **in the same commit**.

---

## 8. Statistics and reporting conventions

- **Paired by seed, always**. Never compare arm means across tranches.
- **Report n after the screen**.
- **Three outcomes for a threshold test, not two**: clears / does not clear / **straddling (inconclusive)**. Report the third as its own verdict rather than rounding it to whichever side you prefer.
- **The adoption rule is one-sided on the point estimate** and is separate from the CI. State both.  $-0.002431$  does not satisfy  $-0.002614 \rightarrow$  no adoption, *even though* the CI does not exclude a true effect above the floor.
- **Token-law control variate**: subtract the throughput-explained component ( $-0.06 \ln(\text{steps}_t / \text{steps}_c)$ ) to cut contention noise  $\sim 6.5\times$  — **for contention only**. **Never for a treatment whose own mechanism is throughput**, or you subtract the effect you are measuring.
- **Check the measured sd against the registered** . In the final block the measured paired sd ( $0.001451$ ) exceeded the registered ( $0.001307$ ) by  $1.11\times$ ; on a recomputed floor the effect sits at  $0.81$

gates rather than 0.90 — *further* from clearing. **Record facts that move against your preferred result.**

- **Cross-scope numbers are not comparable.** The campaign spanned many reconciliations; `val_bpb` values from different scope keys (0.9488, 0.9363, 0.9271) are different questions. Never put them on one axis.

---

## 9. Hard rules — the checklist

Keep this open. Every line is a bug that was hit.

**Before proposing** 1. Re-read the frame arithmetic table. Intrinsic  $\sim 1e-3$  vs gate  $2.6e-3$ . 2. Classify KNOB vs MECHANISM and write it down. 3 zero-adoption blocks means MECHANISM-only. 3. Read one new primary source per block; record the scope arithmetic, yes or no. 4. Check whether the repo already implements it.

**Before launching** 5. Trace the knob to the parameters it modifies; state the % of model touched. 6. Write the prediction, the numeric falsifier, and the trap-check — before data. 7. Selection and confirmation never share draws. 8. `validate` green — and read the **singular error** key. 9. Reconcile the `train.py` hash after any edit; `authorize_run` fails closed. 10. Smoke-test the exact launcher at  $\sim 30$  steps. 11. Write the launcher to a file and `scp` it. **Never heredoc through a single-quoted ssh command** — an apostrophe in the prose has broken this  $3\times$ . 12. Probe-gate the GPUs with a real 40 s training job. Leave 1–2 free. 13. One tranche at a time. Interleave treatment/control across GPUs. 14. **Never edit `train.py` while a tranche is running.** 15. Never run a cron/loop while working interactively.

**When collecting** 16. Screen two-sided against the **stored clean reference**, before looking at `val_bpb`. Report duty and median step for every arm. 17. If the tranche median disagrees with the reference by  $>15\%$ , the tranche is majority-contaminated and the reference wins. 18. Report `n` after the screen.

**When writing up** 19. Commit messages via `git commit -F file`, never inline `-m`. 20. Record the negative, the quarantine reason, and the cost. 21. Update `campaign_log.jsonl`, the registries **and** `research/ideas/` — the last has no automated consumer and will silently rot. 22. Build the PDF. Commit source + PDF together. 23. Republish the chart after **every** experiment, never batched. 24. On adoption: snapshot, then **execute the snapshot standalone.**

---

## 10. Seeding the new project

### 10.1 What to carry over

**Carry the tooling** — it is the part with proven value:

```
tools/registry_authoring.py    # seal() + atomic append(); refuses invalid enums
tools/ssh_retry.sh            # 8 attempts, linear backoff
~/setup/probe_gate.sh         # 40s real-training GPU probe
vibeautoresearch/             # the registry schema + validate/audit gates
make_chart.py                 # chart, republished after every experiment
```

**Carry the four mechanisms** (§3) as MechanismRecords with their causal chains and their *unrun* discriminating predictions.

**Carry this document and `docs/agent_correct_behaviors.md`.**

**Do NOT carry** the 1109-row `campaign_log.jsonl`, the 46 blocks of phase names, or the `research/_legacy_v3_backup/` tree. That is the mess. Start the log clean and cite this document for history.

## 10.2 Day-one checklist

1. **Decide the frame and write its arithmetic table into README.md.** This is the highest-leverage decision (§1.3). If the goal is a wall-clock speedrun, the time-matched frame is right and §5.1 applies immediately.
2. **Re-measure the noise floor on the new host** — 10 control seeds. Do not inherit `2`. Every gate is `2` of *this* box.
3. **Store clean reference step-time medians per configuration** before any A/B.
4. **Stand up validate/audit and make the first commit green.** A control plane discovered red mid-campaign costs a day.
5. **Write paper 001 before experiment 001**, with Next Experiments in the §7.2 format.
6. **Run the three inherited open experiments first (§11)** — they are designed, costed, and have kill criteria.

## 10.3 The one strategic recommendation

Both campaigns hit the same wall from opposite directions: **the frozen corpus**. v3 hit it as an order ceiling (69.9% singleton 5-gram contexts); the reimplementer hit it as a repetition cliff (2.3 passes). Neither could test the fix because the corpus is in both scope keys.

**If the new project can unfreeze the data, do that before anything else.** It converts the single best-evidenced finding of both campaigns from an explanation into a testable prediction, and it is the only intervention that changes the *shape* of the feasible region rather than moving within it.

---

## 11. Open questions worth inheriting

**N1 — Unfreeze the corpus** (out of frame; needs a new scope key) - *Claim*: the frozen ~250M-token corpus, not any architectural choice, is the binding constraint. - *Mechanism*: `mech_data_repetition_cliff`. Tokens = throughput × budget; with a fixed corpus more tokens means more **passes**; past ~2.3 passes repetition inverts the token law. - *Hypothesis*: at a 3× corpus the token law holds with b 0.06 out to 2× the current budget and the cliff does not appear. - *Kill*: if the cliff reappears at the same **pass** count, the mechanism is refuted and the single shuffle draw was unrepresentative. - Ratings — novelty 2, provenance 2, validity 5, impact 5, reliability 4, feasibility 1, falsifiability 5.

**N2 — Count-conditioned n-gram decay** (~1.7 GPU-h) - *Claim*: a penalty that distinguishes memorising rows from statistic-carrying rows closes the gap **and** improves the endpoint. - *Mechanism*: `mech_memoriser_is_the_model`. Norm cannot separate the two row populations; **visit count can**, and it is already computed every step as part of the gather. - *Hypothesis*: decay only rows below visit-count threshold  $c^*$ ; an **interior optimum in  $c^*$**  is the discriminating signature. - *Kill*: a monotone  $c^*$  curve means the count signal is inert and this is just weaker uniform decay, and the weight-space family closes on evidence. - Ratings — novelty 4, provenance 2, validity 4, impact 3, reliability 3, feasibility 5, falsifiability 5.

**N3 — Per-sequence offset augmentation** (~1.7 GPU-h) - *Claim*: our null is evidence about our sampling choice, not about the method. - *Mechanism*: `mech_label_offset_augmentation_identifiability`. Per-token offsets make the target a mixture and the Bayes-optimal output a blur; per-sequence offsets are inferable from context and keep the target identifiable. Label-only methods are the **entire** admissible set, because masking and permutation change token IDs and therefore the n-gram hash keys that are 96.8% of the model. - *Hypothesis*: per-sequence is flat in T over {2,4} where per-token degrades monotonically. - *Kill*: if per-sequence is also null at n=10, the augmentation family closes and N1 is the only move left. - Ratings — novelty 3, provenance 4, validity 4, impact 3, reliability 4, feasibility 5, falsifiability 5.

**Also unrun and cheap**: `obs_val_nats_by_freq_decile` under uniform n-gram decay. If the loss increase

is **uniform across frequency deciles**, the row populations are not frequency-separated and **N2 cannot work** — a one-run precondition that should be checked before N2 is funded.

**Note the tension honestly:** N3 rates highest and N1 matters most. The best-scoring experiment is the cheapest one, and the experiment that would actually change the answer is the one the frame forbids. **If N2 and N3 both die, the recommendation is not a 32nd direction — it is a new frame.**

---

## 12. Closing

A campaign of nulls is not necessarily a campaign of bad ideas. The reimplementation’s null pattern has a real structural cause, and the one lever that moved it was found not by a new idea but by **noticing that a cell of the campaign’s own grid was empty**.

But the deeper lesson is the one in §5.9. The apparatus for *rejecting* claims got very good — four screen revisions, probe gating, atomic registry writes, a two-layer integrity gate, retraction executed on schedule. The apparatus for *generating* claims worth testing never improved at the same rate. **A third campaign that inherits only the rejection machinery will reject faster and find nothing.** Inherit the machinery, and then spend the saved time on the part that was actually scarce: deciding what question the frame is capable of answering, and changing the frame when the answer is “none”.

---

*Sources: vibeaautoresearch-baiyu-v3/research/CAMPAIGN\_SUMMARY.md; vibeaautoresearch\_reimplementation/ — SOTA\_LOG.md, campaign\_log.jsonl (1109 entries), docs/EXPERIMENT\_WORKFLOW.md, docs/AGENT\_PROTOCOL.md, docs/CAMPAIGN\_REFLECTION\_20260731.md, docs/agent\_correct\_behaviors.md (2581 lines), research/ideas/mechanism (25 records), and papers 001–033. All numbers are quoted from those records; none are estimated or reconstructed.*